

Ejercicio 1: Visualización del estado de saturación y pedidos

¿Qué nos piden? Mostrar en la tarjeta de cada restaurante el número de pedidos que ha recibido en las últimas 2 horas. Además, si el restaurante está marcado por el backend como cerrado por límite de pedidos (`isClosedByLimit` es `true`) pero **no** goza del privilegio de ser ilimitado, se debe mostrar visualmente una etiqueta o *badge* con el texto “Saturado”.

Líneas de código que lo solucionan y por qué:

1. Modificación en `src/screens/restaurants/RestaurantsScreen.js` (Renderizado de la tarjeta)

```
<View style={{ marginTop: 10 }}>
  <TextRegular>
    Pedidos (últimas 2h):{" "}
    <TextSemiBold textStyle={{ color: GlobalStyles.brandPrimary }}>
      {item.ordersInLastTwoHours}
    </TextSemiBold>
  </TextRegular>
  {item.isClosedByLimit && !item.isUnlimited && (
    <View style={styles.saturadoBadge}>
      <TextSemiBold textStyle={{ color: "white" }}>Saturado</TextSemiBold>
    </View>
  )}
</View>
```

- `View style={{ marginTop: 10 }}`: Se crea un contenedor nuevo debajo de los costes de envío con un margen superior para mantener la jerarquía visual de la tarjeta (`ImageCard`).
- `{item.ordersInLastTwoHours}`: Se inyecta directamente la propiedad virtual devuelta por el backend usando un componente `TextSemiBold` y el color principal de la marca (`GlobalStyles.brandPrimary`) para que el número destaque sobre el texto regular.
- `{item.isClosedByLimit && !item.isUnlimited && (...)}`: Esta es la **lógica de negocio clave renderizada de forma condicional**. Un restaurante puede tener `isClosedByLimit` a `true` (porque superó el umbral de pedidos), pero si el propietario lo marcó como ilimitado (`item.isUnlimited === true`), la condición `!item.isUnlimited` se evalúa como falsa y el *badge* de “Saturado” se oculta, cumpliendo exactamente con la premisa de que los ilimitados ignoran la saturación.

2. Modificación en `src/screens/restaurants/RestaurantsScreen.js` (Estilos)

```
saturadoBadge: {
  backgroundColor: GlobalStyles.brandPrimary,
  borderRadius: 5,
  paddingHorizontal: 8,
  paddingVertical: 4,
  alignSelf: 'flex-start',
  marginTop: 5
}
```

- `backgroundColor: GlobalStyles.brandPrimary`: En DeliverUS, `brandPrimary` suele ser un color de acento o alerta (rojo/naranja), ideal para indicar un estado de “Saturación” o “Peligro”.
- `**borderRadius, paddingHorizontal/Vertical**`: Convierten un simple texto en una “píldora” o *badge* visual.
- `alignSelf: 'flex-start'`: Evita que el *badge* ocupe todo el ancho de la tarjeta, ajustándose únicamente al ancho del texto “Saturado”.

Ejercicio 2: Botón para conmutar estado ilimitado

¿Qué nos piden? Añadir un botón interactivo a cada restaurante que permita al propietario activar o desactivar su estado “ilimitado”. Esto requiere una petición a la API y que la interfaz cambie dinámicamente según el estado en el que se encuentre el restaurante.

Líneas de código que lo solucionan y por qué:

1. Modificación en src/api/RestaurantEndpoints.js (Nueva llamada a la API)

```
/* SOLUTION */
function toggleIsUnlimited (id) {
  return patch(`restaurants/${id}/toggleIsUnlimited`)
}
export { ..., toggleIsUnlimited }
```

- ****Uso del verbo patch****: Se exporta una nueva función que invoca al helper HTTP patch. Es el método semánticamente correcto porque solo estamos mutando una propiedad específica (isUnlimited) del recurso Restaurant, no reemplazando el objeto entero (que sería un put).

2. Modificación en src/screens/restaurants/RestaurantsScreen.js (Importación)

```
import {
  getAll,
  remove,
  toggleIsUnlimited,
} from "../../api/RestaurantEndpoints";
```

- Inyección de la dependencia recién creada en el controlador de la vista.

3. Modificación en src/screens/restaurants/RestaurantsScreen.js (Botón UI en renderRestaurant)

```
<Pressable
  onPress={() => handleToggleUnlimited(item)}
  style={({ pressed }) => [
    {
      backgroundColor: pressed
        ? item.isUnlimited
          ? GlobalStyles.brandSecondaryTap
            : GlobalStyles.brandSuccessTap
        : item.isUnlimited
          ? GlobalStyles.brandSecondary
            : GlobalStyles.brandSuccess,
    },
    styles.actionButton,
  ]}
>
<View style={[{ flex: 1, flexDirection: "row", justifyContent: "center" }]}>
  <MaterialCommunityIcons
    name={item.isUnlimited ? "infinity-off" : "infinity"}
    color={"white"}
    size={20}
  />
  <TextRegular textStyle={styles.text}>
    {item.isUnlimited ? "Limited" : "Unlimited"}
  </TextRegular>
</View>
</Pressable>
```

- **onPress={() => handleToggleUnlimited(item)}**: Lanza la función asíncrona pasando el objeto del restaurante como contexto.
- **Lógica del color de fondo (Doble operador ternario)**:
 - Si el botón *no está presionado*: evalúa `item.isUnlimited`. Si ya es ilimitado, se pinta de gris (`brandSecondary`) para indicar que la acción a realizar será “apagarlo”. Si es limitado, se pinta de verde (`brandSuccess`) para indicar que al pulsarlo se le dará una mejora (hacerlo ilimitado).

- Si el botón *está presionado*: hace exactamente lo mismo pero aplicando las variables con sufijo `Tap`, lo que proporciona *feedback* háptico/visual nativo al usuario (el botón se oscurece al tocarlo).
- **Iconografía dinámica** (`name={item.isUnlimited ? 'infinity-off' : 'infinity'}`): Si es ilimitado, muestra el icono de infinito tachado (acción de desactivar). Si no, muestra el icono de infinito.
- **Texto dinámico**: Muestra 'Limited' (para volverlo a la normalidad) o 'Unlimited' (para potenciarlo).

Ejercicio 3: Gestión de error por límite de restaurantes ilimitados

¿Qué nos piden? Manejar la comunicación con el backend cuando se pulsa el botón del RF.02. Si la petición falla (específicamente por el error 409 cuando se intentan tener más de 3 restaurantes ilimitados), se debe interceptar el error y mostrar un mensaje descriptivo sin romper la aplicación.

Líneas de código que lo solucionan y por qué:

1. Modificación en `src/screens/restaurants/RestaurantsScreen.js` (Controlador de estado)

```
/* SOLUTION */
const handleToggleUnlimited = async (restaurant) => {
  try {
    await toggleIsUnlimited(restaurant.id);
    await fetchRestaurants();
    showMessage({
      message: `Estado ilimitado actualizado para ${restaurant.name}`,
      type: "success",
      style: GlobalStyles.flashStyle,
      titleStyle: GlobalStyles.flashTextStyle,
    });
  } catch (error) {
    console.log(error);
    showMessage({
      message: `Error: Un propietario no puede tener más de 3 restaurantes ilimitados.`,
      type: "error",
      style: GlobalStyles.flashStyle,
      titleStyle: GlobalStyles.flashTextStyle,
    });
  }
};
```

- ****Bloque try/catch****: Esencial para manejar promesas rechazadas generadas por el helper de Axios cuando el servidor devuelve un código HTTP distinto a 2xx (en este caso, un 409 Conflict).
- **await toggleIsUnlimited(restaurant.id)**: Bloquea la ejecución de la función hasta que el backend procesa el cambio en la base de datos y responde afirmativamente.
- **await fetchRestaurants()**: **Línea crítica**. En lugar de intentar mutar el estado local de React (`setRestaurants`) de forma manual (lo cual es propenso a errores de desincronización), se vuelve a solicitar la lista completa de restaurantes al servidor. Esto actualiza el `state` desencadenando un re-renderizado automático del `FlatList`, asegurando que la UI refleje el nuevo estado `isUnlimited` de forma 100% veraz.
- ****showMessage en el try****: Llama a la librería `react-native-flash-message` con `type: 'success'` (color verde) dando feedback positivo de que la acción tuvo efecto.
- ****showMessage en el catch****: Si el backend rechaza la petición (por superar el límite de 3), el código salta directamente aquí. Al no ejecutarse el `fetchRestaurants()`, la interfaz no muta el botón incorrectamente. Se dispara el mensaje con `type: 'error'` (color rojo) especificando exactamente la regla de negocio que se ha violado (“Un propietario no puede tener más de 3...”), lo que cumple a rajatabla el RF.02 de control de saturación. Se mantiene la consistencia visual y de estado.